BENILDE ANTIPOLO

FLORES, KURT JUSTINE B.
BSIS01 DSTALGO
Data Structures and Algorithm

Week 6 Research: Data Structures and Algorithms

1. Linear Data Structure

Definition: A data structure is *linear* when its elements are arranged one after another in a single, orderly sequence, so all elements can be visited in one pass. Each element has a defined predecessor and successor, except possibly the first and last. This makes searching, iterating, and updating simpler. Arrays and singly linked lists are examples. Others include stacks and queues. Trees and graphs are *non-linear* because their elements are not stored in a single sequence (Sharma, 2026).

Why it matters: Because elements sit in one sequence, they can be visited in a single pass. This makes searching, iterating, and updating straightforward.

One-Dimensional Array

- Definition: The one-dimensional (linear) array is the simplest data structure. Its elements are numbered with consecutive integers (indexes), and each element is accessed by its index (Encyclopaedia Britannica, n.d.).
- Key characteristics:
  - It is homogeneous: all elements have the same data type.
  - It is ordered and finite, and its elements are stored in consecutive memory locations.
  - Each element is reached through a single index, as in array[3].
  - Its size is usually fixed once it is created.
- Example (C): int a[5]; declares an array that holds 5 integers, indexed 0 to 4.
- Cost of common operations:
  - Access by index takes constant time.
  - Adding or removing an element at the end is typically fast.
  - Inserting or deleting in the middle requires shifting elements, which takes more time.

2. Big O Notation

Definition: Big O notation is a convenient way to describe how fast a function is growing (Huang, 2020). In computer science it describes how an algorithm's running time (time complexity) or memory use (space complexity) grows as the input size *n* grows. It is used to compare the worst-case scenarios of algorithms (Ndlovu, 2023).

Simple meaning: If T(n) is O(f(n)), then T(n) does not grow faster than f(n). Big O ignores constants and small details and focuses on the overall growth trend.

Common complexities (from fastest to slowest growth):

Notation	Name	Meaning	Example
O(1)	Constant	Time stays the same regardless of input size	Accessing an array element by index (Yangani, 2015)
O(log n)	Logarithmic	Time grows very slowly	Binary search (GeeksforGeeks, 2025)
O(n)	Linear	Time grows in proportion to input size	Linear search (GeeksforGeeks, 2025)
O(n log n)	Linearithmic	Between linear and quadratic	Merge sort, quick sort
O(n²)	Quadratic	Time grows with the square of the input	Bubble sort, nested loops (Yangani, 2015)

Note: Big O gives a growth trend, not an exact running time. It typically ignores constant factors, so two algorithms with the same complexity can run at very different real speeds.

3. Basic Searching Algorithms

A searching algorithm finds a specific item in a collection of data.

Linear Search (Sequential Search)

- **Definition:** A linear search scans each element of the array in order until the required element is found. Its worst-case time is O(n), because the time needed grows with the number of elements (Educative, n.d.). The worst case happens when the target is absent and the whole array must be checked. It uses only O(1) extra space, and the data does not need to be sorted (GeeksforGeeks, 2025).
- **How it works:**
  1. Start at the first element.
  2. Compare it with the target.
  3. If it matches, return its position. Otherwise move to the next element.
  4. If the end is reached without a match, report "not found."
- **Sorted data required?** No.
- **Time complexity:** O(n) in the worst case, which happens when the target is not in the array and every element must be checked. The space complexity is O(1).
- **Best for:** small or unsorted collections.

Binary Search (Half-Interval Search)

- **Definition:** Binary search is a divide-and-conquer algorithm that works on sorted data. It repeatedly halves the search space until the target is found or shown to be absent.
- **How it works:**
  1. Set **low** to the first index and **high** to the last index.
  2. Find the middle element.
  3. If it equals the target, stop, return its position.
  4. If the target is smaller, search the left half. If it is larger, search the right half.
  5. Repeat until the target is found or **low > high** (not found).
- **Sorted data required?** Yes.
- **Time complexity:** O(log n). The space complexity is O(1) because only the low, high, and mid indices are stored.

- **Best for:** large, sorted collections.

COMPARISON

Feature	Linear Search	Binary Search
Data must be sorted?	No	Yes
Time complexity (worst case)	O(n)	O(log n)
Method	Checks items one by one	Halves the search area each step
Complexity to implement	Simple	Slightly more complex
Best for	Small or unsorted data	Large sorted data

(Adapted from Educative, n.d.; GeeksforGeeks, 2025.)

Example: In a sorted array of 1,000,000 elements, binary search needs about 20 steps ($\log_2 1{,}000{,}000 \approx 20$), while linear search may need up to 1,000,000.

Trade-off: Binary search needs sorted data. Sorting typically costs O(n log n), so if you only search once, linear search can be cheaper overall. If you search many times, sorting first pays off (Klein, n.d.).

REFERENCES

Educative. (n.d.). *Linear vs. binary search*. https://www.educative.io/answers/linear-vs-binary-search

Encyclopaedia Britannica. (n.d.). *Linear array*. https://www.britannica.com/technology/linear-array

GeeksforGeeks. (2025, July 2). *Linear search vs binary search*. https://www.geeksforgeeks.org/dsa/linear-search-vs-binary-search/

Huang, S. (2020, January 16). *What is Big O Notation Explained: Space and Time Complexity*. freeCodeCamp.org. https://www.freecodecamp.org/news/big-o-notation-why-it-matters-and-why-it-doesnt-1674cfa8a23c/

Klein, R. (n.d.). *Chapter 3 Searching and Sorting | Introduction to Data Structures & Algorithms*. https://courses.ms.wits.ac.za/~richard/IDSA-Notes-YouTube/sorting.html

Ndlovu, V. (2023, April 28). *An overview of Big O notation*. DEV Community. https://dev.to/vndlovu/an-overview-of-big-o-notation-4j88

Sharma, R. (2026, July 6). *What is Linear Data Structure and its Types? Explore Differences With Nonlinear Structures*. upGrad Blog. https://knowledgehut.com/blog/programming/linear-data-structure

Yangani, F. K. (2015, October 12). *A beginner's guide to Big O notation*. freeCodeCamp. https://medium.freecodecamp.org/my-first-foray-into-technology-c5b6e83fe8f1